



Desarrollo con Spring Boot

```
01 @SpringBootApplication
02 public class Aplicacion {
03     public static void main(String[] args) {
04         SpringApplication.run(Aplicacion.class, args);
05     }
06 }
07
08 @RestController
09 @RequestMapping("/api")
10 public class SaludoController {
11     @GetMapping("/saludo")
12     public String saludo() {
13         return "¡Hola desde Spring Boot!";
14     }
15 }
```



IoC



MVC



REST



JPA



IoC. Inyección de dependencias. Beans.
MVC. REST APIs. Spring Data. Seguridad.



INYECCIÓN DE
DEPENDENCIAS



SPRING MVC



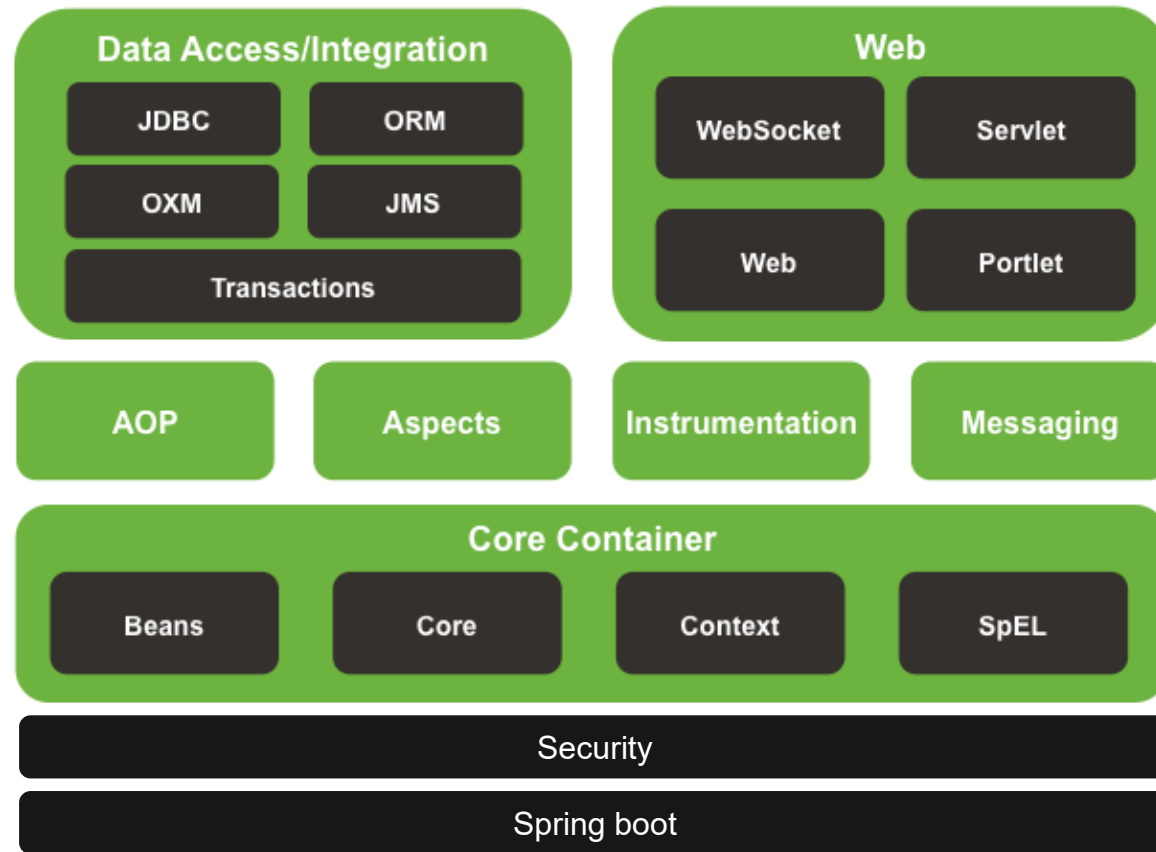
REST APIs



SPRING DATA JPA



SPRING FRAMEWORK



Spring Framework: módulos principales

Spring Framework ofrece los módulos base. Spring Boot simplifica su uso mediante autoconfiguración, starters y configuración externalizada

spring-core	Contenedor IoC, beans, inyección de dependencias
spring-context	ApplicationContext, eventos, internacionalización, configuración
spring-beans	Definición, creación y ciclo de vida de beans
spring-aop	Programación orientada a aspectos
spring-expression	SpEL, expresiones dinámicas
spring-jdbc	Acceso JDBC simplificado
spring-tx	Gestión declarativa de transacciones
spring-orm	Integración con JPA/Hibernate
spring-web	Infraestructura web
spring-webmvc	Modelo MVC clásico
spring-test	Testing unitario e integración



Tipos de Aplicaciones

Spring permite la creación de todo tipo de aplicaciones: aplicaciones web, aplicaciones multicapa, clientes ricos.

- **No es invasivo:** permite su uso conjunto a otros frameworks (p.e.: Struts o Hibernate).
- **No es todo o nada:** no obliga a la implementación de toda la aplicación empleando Spring.
- **Aplicaciones empresariales “ligeras” y basadas en Java EE:** no requieren de un servidor de aplicaciones, solamente una máquina virtual. Pero también se pueden integrar con servidores JEE para obtener las ventajas que estos ofrecen (Seguridad frente a fallos, escalabilidad, etc).
- **Sistema declarativo de manejo de transacciones:** basado en AOP puede manejar de forma declarativa las de un único recurso transaccional (p.e.: un DataSource). Si se desean realizar transacciones globales (que agrupen varios recursos transaccionales) se deberá usar una transacción de un servidor JEE mediante JTA.

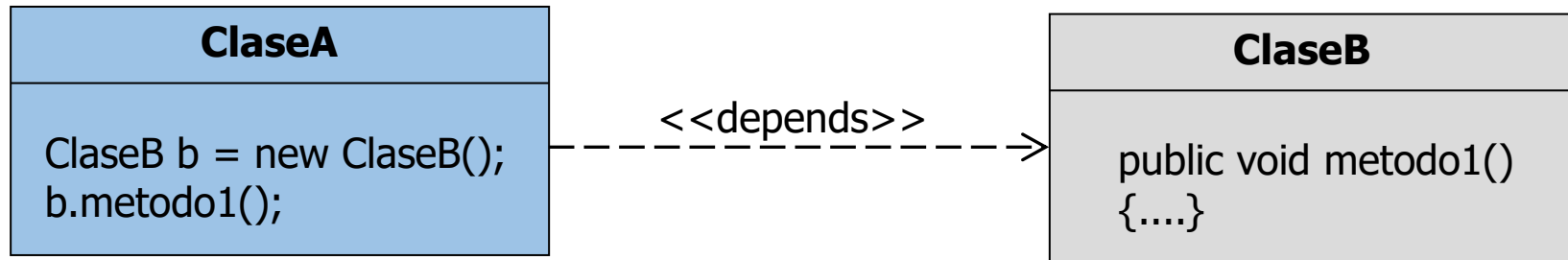
SECCIÓN

Dependencias

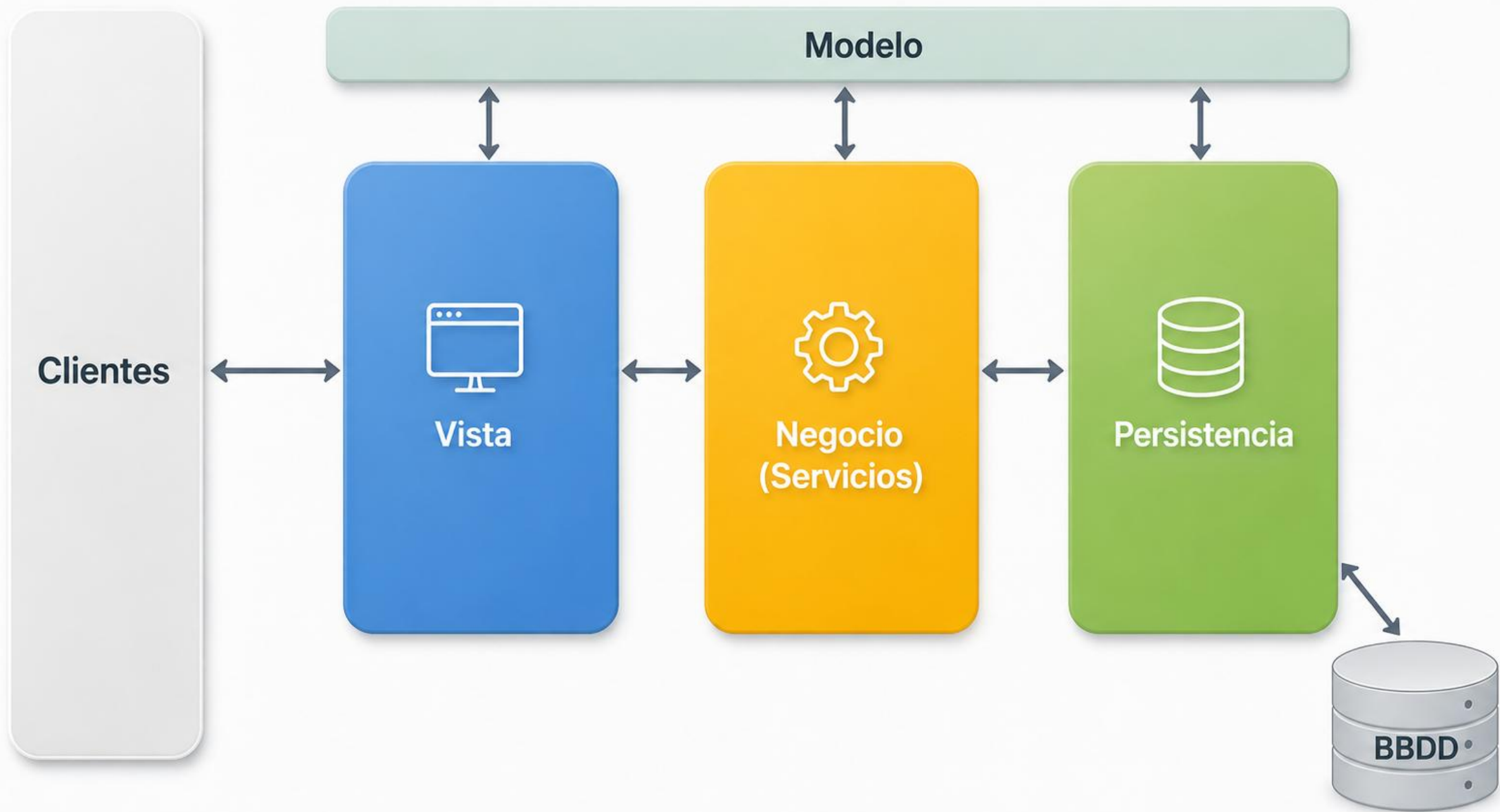


Dependencias

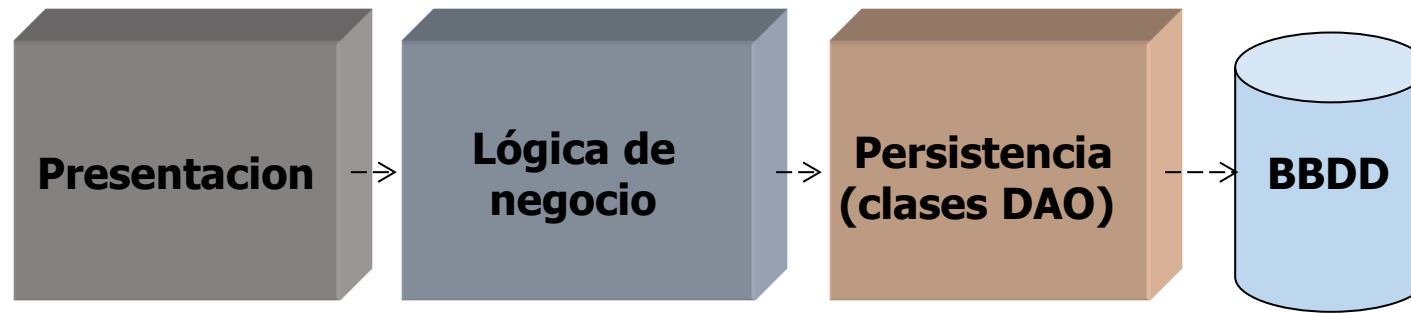
Se dice que una clase A depende de otra B cuando A tiene referencias de tipo B y crea objetos de B



- 1.- Difícil mantenimiento y rediseño: En tiempo de diseño, cualquier cambio realizado en una clase, puede afectar a las clases que dependen de ella y por lo tanto que sea necesario modificarlas también.
- 2.- Menor estabilidad: En tiempo de ejecución, si una parte de la aplicación tiene un error o deja de funcionar, si el sistema tiene muchas dependencias, se verá mayormente afectado.

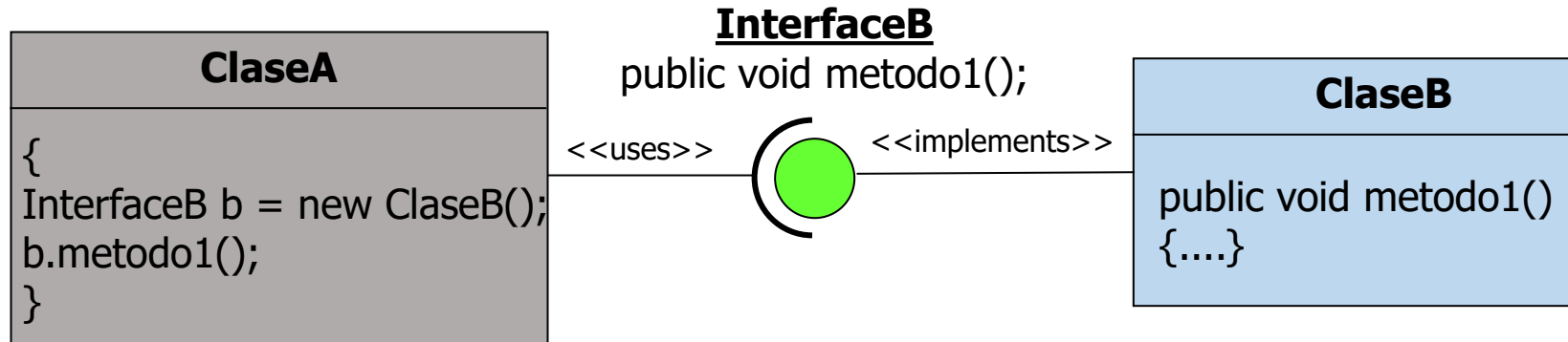


Reducir el Acoplamiento



- Cada capa solamente se comunica con las capas contiguas y por lo tanto solamente depende de ellas.
- Ej: No hay dependencia entre la capa de Presentación y la de Persistencia

Reducir el acoplamiento - Interfaces



Con el uso de interfaces, las referencias empleadas en una parte de la aplicación no quedan acopladas a una clase (implementación concreta), sino a una "interface".

SECCIÓN

Inyección de Dependencias



Reducir el Acoplamiento – Inyección de Dependencias (DI)

ClaseA
<pre>{ InterfaceB b = new ClaseB(); b.metodo1(); }</pre>

DI por parámetro

ClaseA
<pre>InterfaceB b; public void setB(InterfaceB b){ this.b=b; } ... b.metodo1();</pre>

DI por constructor

ClaseA
<pre>InterfaceB b; public ClaseA (InterfaceB b){ this.b=b; } ... b.metodo1();</pre>

Creación de un bean

```
@Configuration
public class A04Config {

    @Bean(name="unaPersona")
    public PersonaInterface getPersona(){
        Persona p = new Persona();
        p.setId(23);
        p.setNombre("Pedrito");
        p.setTelefono("1234546778");
        return p;
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/ ...>

    <bean id="unaPersona" class="agenda.Persona">
        <property name="id" value="23"/>
        <property name="nombre">
            <value>Pedrito</value>
        </property>
        <property name="telefono" value="1234546778"/>
    </bean>

</beans>
```

Inyección de dependencias (DI)

```
@Bean(name="unPadre")
public PersonaInterface padre(){
    Persona p = new Persona();
    p.setNombre("Pepe");
    return p;
}

@Bean(name="unaMadre")
public PersonaInterface madre(){
    Persona p = new Persona();
    p.setNombre("Ana");
    return p;
}
```

```
@Bean(name="unaFamilia")
public Familia getFamilia(){
    Familia f = new Familia();
    f.setPadre(padre());
    f.setMadre(madre());
    return f;
}
```

```
<bean id="unPadre" class="agenda.Persona">
    <property name="nombre" value="Pepe" />
    <property name="telefono" value="1234546778"/>
</bean>

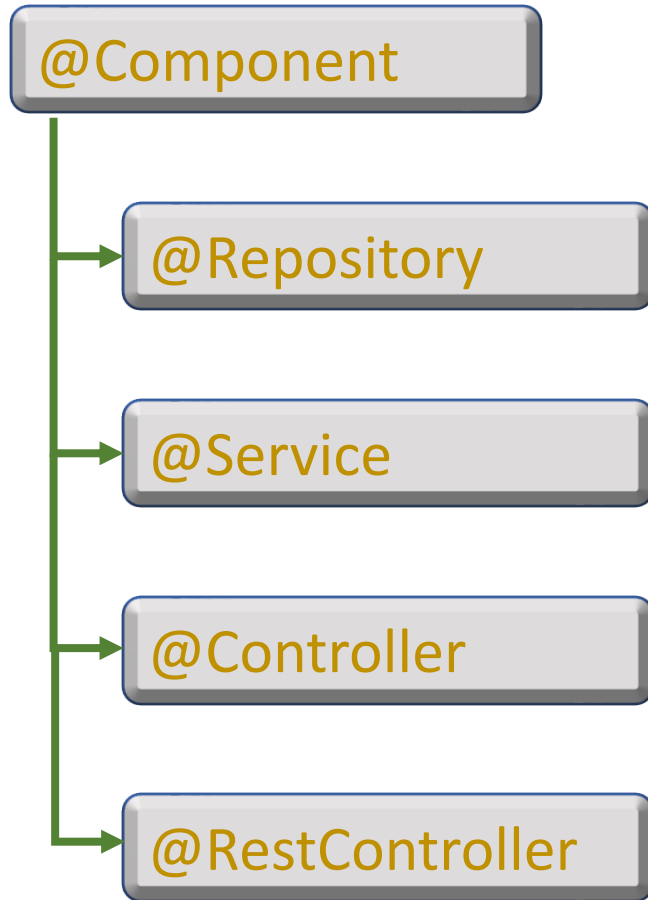
<bean id="unaMadre" class="agenda.Persona">
    <property name="nombre" value="Ana" />
    <property name="telefono" value="987654321"/>
</bean>

<bean id="unaFamilia" class="agenda.Familia">
    <property name="padre" ref="unPadre" />
    <property name="madre" ref="unaMadre" />
</bean>
```

```
@Component
public class Familia {
    private Persona padre;
    private Persona madre;

    @Autowired
    public Familia(Persona padre, Persona madre) {
        this.padre = padre;
        this.madre = madre;
    }
}
```

@Component (stereotype annotations)



```
@Component  
public class MiComponenteGenerico { }
```

```
@Service  
public class UsuarioService { }
```

```
@Repository  
public class UsuarioRepository { }
```

```
@Controller  
public class UsuarioController { }
```

```
@RestController  
public class UsuarioRestController { }
```

SECCIÓN

Spring Boot



SECCIÓN

Soporte a la configuración



Soporte de configuración en Spring Boot

- Spring Boot permite externalizar configuración para usar el mismo código en distintos entornos
 - application.properties
 - application.yml
 - perfiles: dev, test, prod
 - variables de entorno
 - argumentos de línea de comandos
 - configuración por entorno: application-dev.yml, application-prod.yml
 - @ConfigurationProperties

application.properties

spring.application.name=presentaciones

server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/salas

spring.datasource.username=app

spring.datasource.password=secreta

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

app.sales.capacidad-maxima=30

app.sales.permitir-reservas-externas=true

application.yaml

server:

port: 8081

spring:

datasource:

url: jdbc:mysql://localhost:3306/salas

username: app

password: secreta

driver-class-name: com.mysql.cj.jdbc.Driver

application:

name: presentaciones

app:

salas:

capacidad-maxima: 30

permitir-reservas-externas: true

@ConfigurationProperties

```
@ConfigurationProperties(prefix = "app.salas")
public class SalaProperties {

    private Integer capacidadMaxima;
    private Boolean permitirReservasExternas;

    public Integer getCapacidadMaxima() {
        return capacidadMaxima;
    }
    //...
```

```
@ConfigurationPropertiesScan
@SpringBootApplication
public class PresentacionesApplication {

    public static void main(String[] args) {
        SpringApplication.run(PresentacionesApplication.class, args);
    }
}
```

Perfiles

- application.yml
 - Configuración común a la aplicación
- application-dev.yml
 - Configuración de base de datos y otros recursos en entorno de desarrollo
- application-test.yml
 - Configuración de base de datos y otros recursos en entorno de pruebas
- application-prod.yml
 - Configuración de base de datos y otros recursos en entorno de producción

```
PABLO:~$ export SPRING_PROFILES_ACTIVE=prod
```

```
:~$ java -jar app.jar --spring.profiles.active=prod
```



Variables de entorno

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/salas
    username: app
    password: ${DB_PASSWORD}
    driver-class-name: com.mysql.cj.jdbc.Driver
  application:
    name: presentaciones
```

Linux/maxOS

```
~$
~$ export DB_PASSWORD=mi_password_real
```

```
ubuntu@THINKPAD-PABLO:~$
ubuntu@THINKPAD-PABLO:~$ echo $DB_PASSWORD
mi_password_real
```

Windows (CMD)

```
C:\Users\plope>setx DB_PASSWORD "mi_password_real"
CORRECTO: se guardó el valor especificado.
```

```
C:\Users\plope>echo %DB_PASSWORD%
mi_password_real
```